

Computer Use

从 CLI 到桌面自动化——Claude Code 通过 15 个专用文件构建了一套完整的 Computer Use 子系统，将屏幕截图、鼠标点击、键盘输入等桌面操作纳入 agent 能力面。

核心问题

一个运行在终端里的 CLI 工具，如何安全地获得"看到屏幕并操作桌面"的能力？Computer Use 子系统解决的不只是"能不能点"，而是"谁能点、在哪点、出了问题怎么恢复"。

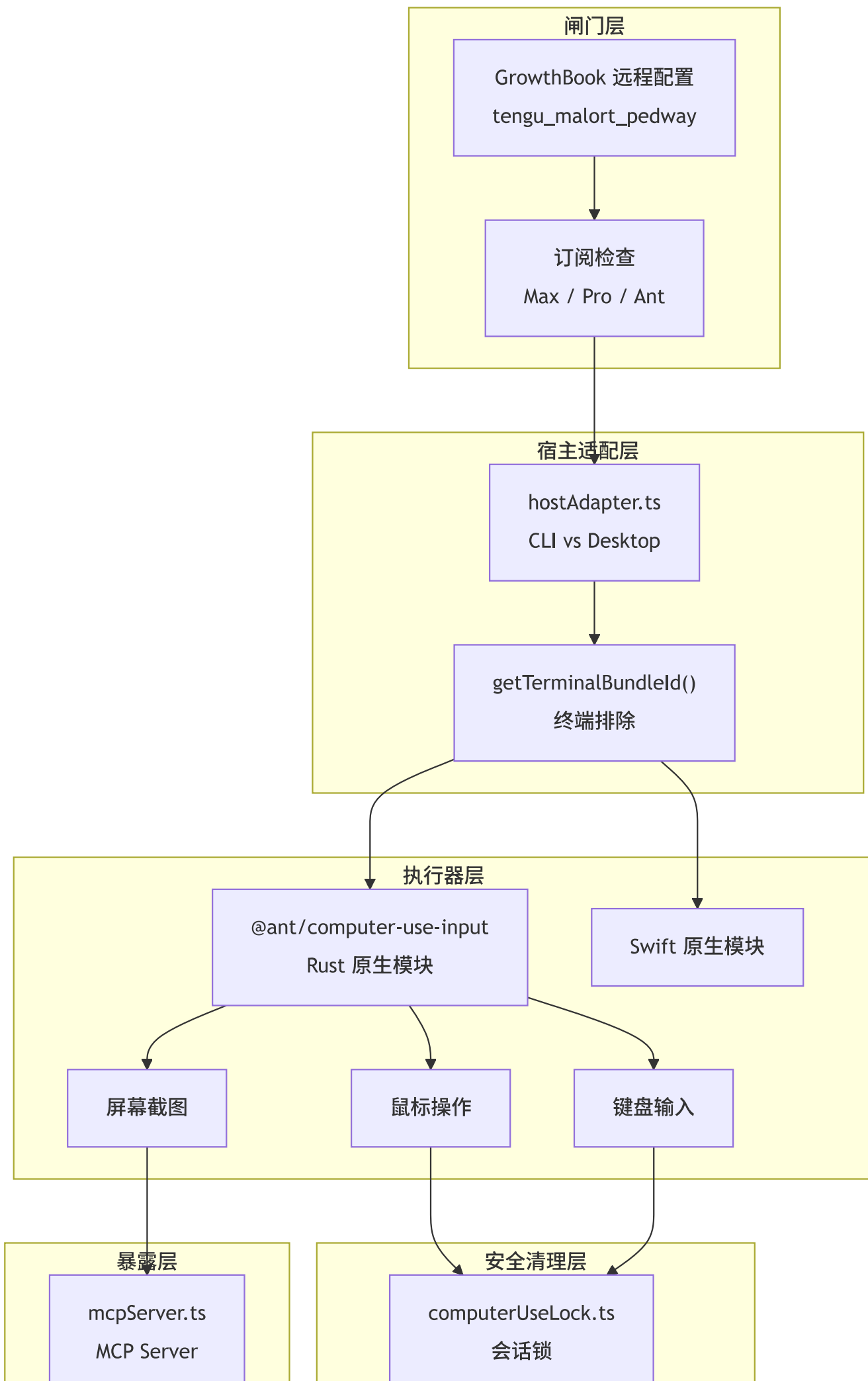
子系统全景

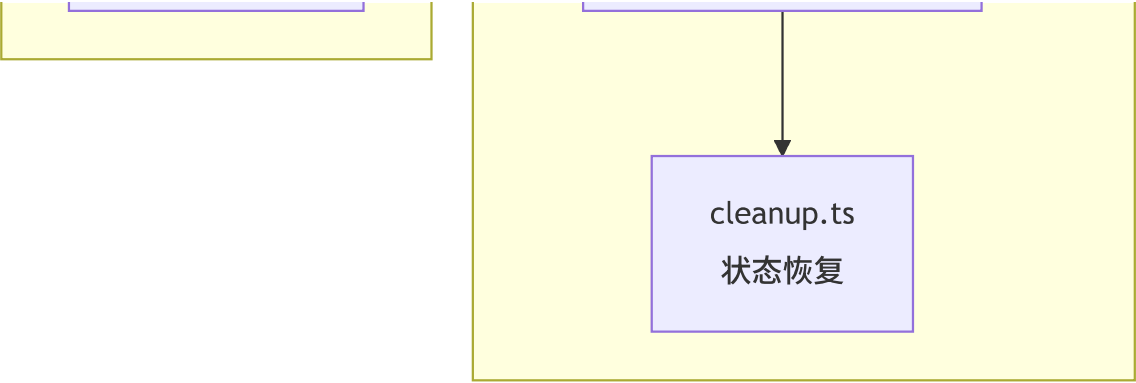
指标	数值
专用文件数	15
目录位置	src/utils/computerUse/
成熟度	Integrated（已集成到主干）
订阅要求	Max / Pro 用户（内部员工绕过）

架构分层

组件	文件	职责
执行器	executor.ts	封装 Rust (@ant/computer-use-input) 和 Swift 原生模块，处理鼠标、键盘、屏幕截图
闸门	gates.ts	通过 GrowthBook 远程配置控制功能开关（代号 <code>tengu_malort_pedway</code> ）
宿主适配	hostAdapter.ts	CLI 与桌面端的差异适配——CLI 作为终端代理运行时需要特殊处理前台窗口
MCP 服务	mcpServer.ts	将 Computer Use 能力作为 MCP Server 暴露给外部消费者

组件	文件	职责
安全清理	<code>cleanup.ts</code> / <code>computerUseLock.ts</code>	会话结束时恢复鼠标/键盘状态，防止中断导致的桌面控制残留
剪贴板	通过 <code>pbcopy</code> / <code>pbpaste</code>	不依赖 Electron，直接调用系统剪贴板





关键设计决策

订阅闸门

`hasRequiredSubscription()` 限制只有 Max/Pro 用户才能使用 Computer Use。Ant（内部员工）绕过检查。这是一种典型的**渐进式能力暴露**——高风险能力优先向付费用户和内部测试者开放。

终端代理模式

CLI 运行在终端模拟器中时，`getTerminalBundleId()` 检测当前运行的终端应用。截图和点击时需要将终端自身从目标中排除——否则 agent 会截到自己的命令行界面，甚至误点自己的窗口。

MCP 暴露

Computer Use 不仅供 Claude Code 内部使用，还通过 `mcpServer.ts` 作为 MCP Server 暴露。外部 MCP 客户端可以消费 Computer Use 能力，这意味着桌面操作可以被编排到更大的 agent 工作流中。

会话锁与状态恢复

`computerUseLock.ts` 实现会话级锁机制。当 Computer Use 会话中断（用户关闭终端、进程崩溃）时，`cleanup.ts` 确保鼠标和键盘状态被恢复到原始状态，防止桌面控制残留。

与治理体系的关系

Computer Use 不是绕过权限系统的"超级能力"。它仍然受到完整的四层治理约束：

- 1. **GrowthBook 闸门**：远程配置可以随时关闭
- 2. **订阅检查**：按用户级别限制
- 3. **权限模式**：工具执行前仍经过标准权限链路
- 4. **沙箱约束**：在沙箱环境中 Computer Use 能力受限

对 agent 开发者的启示

模式	说明
原生模块延迟加载	桌面操作依赖的原生模块（Rust/Swift）不在启动时加载，而是首次使用时才 <code>dlopen</code> ——避免 ~8s 的冷启动阻塞
闸门分层	远程配置 + 订阅检查 + 权限系统三层独立控制，任一层都可以单独关闭能力
宿主感知	agent 操作桌面时必须知道自己"在哪"——终端排除是一个容易被忽略但关键的设计点
安全清理	任何可能改变外部状态的 agent 能力都需要"会话结束恢复"机制

路径证据

路径	职责
<code>src/utils/computerUse/</code>	完整子系统（15 个文件）
<code>src/utils/computerUse/executor.ts</code>	CLI 执行器实现
<code>src/utils/computerUse/gates.ts</code>	功能闸门与远程配置
<code>src/utils/computerUse/hostAdapter.ts</code>	宿主适配层
<code>src/utils/computerUse/mcpServer.ts</code>	MCP Server 暴露
<code>src/utils/computerUse/cleanup.ts</code>	安全清理
<code>src/utils/computerUse/computerUseLock.ts</code>	会话锁

进一步阅读

1. [架构地图](#) – 理解 Computer Use 在六层结构中的位置
2. [工具平面](#) – 看 Computer Use 如何融入工具治理体系
3. [权限治理](#) – 理解闸门和权限的关系
4. [扩展与信号](#) – 查看 Computer Use 的成熟度判定

[Claude 中文](#) [ChatGPT 中文](#) [MCP 中文站](#) [A2A 中文站](#) [ACP 商务协议](#) [AP2 Lab](#)
[Sora AI 中文](#) [UCP 技术](#) [MoltBot 中文](#) [OpenClaw 中文](#)

由 Claude 中文社区驱动

© 2025 Claude 中文社区 [✉ ai@claudecn.com](mailto:ai@claudecn.com)